

École de technologie supérieure

Département de génie logiciel et des TI

Rapport de Laboratoire 3

Dessin de primitives, Remplissage et Soustraction de fond

Cours : GTI411 – Imagerie numérique

Chargée de laboratoire : Jouhaina Nasri

Nom : JONNIAUX

Prénom : Mathieu

Automne 2026

Table des matières

1 Introduction **2**

2 Outils et concepts **2**

2.1 Concepts fondamentaux 2

2.2 Autres cas d'application 3

3 Expérimentations et discussion **3**

3.1 Partie 1 : Dessin de primitives 3

3.1.1 Implémentation du tracé et gestion des octants 3

3.1.2 Comparaison du rectangle : Bresenham vs OpenCV 5

3.1.3 Amélioration de la Partie 1 : prévisualisation en direct 5

3.2 Partie 2 : Remplissage automatisé (BoundaryFill et FloodFill) 6

3.2.1 BoundaryFill récursif et validation sur `test.png` 6

3.2.2 Limite de BoundaryFill sur `cube.png` et passage à FloodFill 7

3.3 Partie 3 : Extraction de la silhouette humaine 8

3.3.1 Logique du pipeline de soustraction de fond 8

3.3.2 Justification et visualisation des étapes intermédiaires 9

3.3.3 Résultats finaux : fond flouté et fond remplacé 11

4 Difficultés et améliorations **12**

4.1 Difficultés rencontrées et limites 12

4.2 Améliorations apportées et proposées 12

5 Conclusion **13**

1 Introduction

Dans le cadre du cours GTI411, ce troisième laboratoire met en pratique deux sujets vus en classe : le tracé de primitives et le remplissage de régions dans une petite application de type « Paint », puis la soustraction d'arrière-plan en temps réel avec une webcam comme sur Zoom ou Teams. Comme pour le laboratoire 2, je n'ai utilisé que des méthodes de traitement d'image classiques vues en cours, aucun modèle d'apprentissage profond.

Le travail se divise en trois parties :

1. **Partie 1 (dessin de primitives)** : coder le tracé d'une ligne avec l'algorithme du point milieu de Bresenham dans toutes les directions, puis tracer un rectangle, un cercle, etc. à partir de deux points cliqués à la souris.
2. **Partie 2 (remplissage automatisé)** : coder à la main l'algorithme récursif `BoundaryFill`, puis utiliser `FloodFill` avec OpenCV en jouant sur les seuils de tolérance *low* et *high*.
3. **Partie 3 (extraction de la silhouette humaine)** : enregistrer une image du décor vide, soustraire ce fond à chaque image de la webcam, nettoyer le masque binaire, puis flouter l'arrière-plan ou le remplacer par une image au choix.

Le code des parties 1 et 2 est organisé en MVC en Python avec PyQt5 comme aux laboratoires précédents, et la partie 3 a sa propre fenêtre pour la webcam. Ce rapport présente d'abord les concepts utilisés, puis la logique et les résultats de chaque partie, et enfin les difficultés rencontrées et les améliorations possibles.

2 Outils et concepts

2.1 Concepts fondamentaux

Voici les notions du cours que j'ai utilisées :

— **Algorithme du point milieu (Bresenham)** : je m'en sers dans la partie 1 pour tracer les lignes, les rectangles et les cercles pixel par pixel seulement avec des additions et soustractions d'entiers, sans faire de calculs à virgule.

— **Remplissage en voisinage 4 (`BoundaryFill` et `FloodFill`)** : j'ai codé `BoundaryFill` en récursif pour s'arrêter sur une couleur de bordure précise, et j'utilise `cv2.floodFill` avec une plage de tolérance pour remplir des zones qui ont des ombres, de la compression, etc. J'utilise 4 voisins au lieu de 8 pour éviter que la couleur fuie en diagonale entre deux pixels d'une ligne de Bresenham.

— **Soustraction de fond et seuillage** : dans la partie 3, je calcule l'écart entre l'image de la webcam et la photo du fond vide avec `cv2.absdiff`, puis j'applique un seuil binaire pour isoler la personne.

— **Filtre gaussien, morphologie et contours** : comme au laboratoire 2, je passe un flou gaussien avant presque tout pour enlever le bruit de la caméra. Une ouverture et une fermeture nettoient ensuite le masque, puis `findContours` garde les grandes régions et bouche les trous à l'intérieur.
— **Alpha-blending** : un flou gaussien sur le bord du masque donne une transparence progressive entre 0 et 1 pour incruster la personne sur le nouveau fond sans contour trop découpé.

Les bibliothèques utilisées sont OpenCV (OpenCV) pour le traitement d'image, NumPy (NumPy) pour les calculs sur les matrices de pixels et PyQt5 (Riverbank Computing) pour l'interface.

2.2 Autres cas d'application

Les techniques utilisées dans ce laboratoire servent aussi dans d'autres domaines :
— **Logiciels de dessin** : Paint, Photoshop, etc. utilisent un `FloodFill` avec tolérance.
— **Écrans embarqués** : sur des montres connectées ou de petits microcontrôleurs peu puissants, Bresenham permet d'afficher des formes très vite sans calcul flottant.
— **Visioconférence et fond vert** : Zoom ou Teams utilisent la soustraction de fond et l'alpha-blending pour flouter une pièce ou changer le décor en direct.
— **Caméras de surveillance** : comparer le flux vidéo à une image du lieu vide permet de repérer automatiquement si quelqu'un entre dans une zone ou si quelque chose à changé.

3 Expérimentations et discussion

3.1 Partie 1 : Dessin de primitives

3.1.1 Implémentation du tracé et gestion des octants

Dans le cours, l'algorithme de Bresenham est d'abord donné pour une ligne qui monte doucement de gauche à droite. Mais dans l'application, on peut tirer la souris dans n'importe quel sens. Plutôt que d'écrire 8 versions du code pour les 8 octants, je me ramène toujours au cas de base avec trois vérifications :
— Si la ligne est plus verticale qu'horizontale, j'échange x et y au début et je les remets dans le bon ordre au moment d'allumer le pixel.
— Si on tire la souris vers la gauche, j'échange le point de départ et le point d'arrivée pour toujours avancer avec x croissant.
— Si la ligne descend, le pas vertical passe à -1 au lieu de $+1$.

Pour le cercle, les consignes entre le document Word et le PowerPoint ne disaient pas exactement la même chose sur le droit d'utiliser OpenCV. Dans le doute, j'ai codé l'algorithme du point milieu à la main avec les 8 symétries vues en cours, en prenant la distance entre `start_point` et `end_point` comme rayon. Pour l'ellipse, qui n'était pas vue en détail en cours, j'ai utilisé `cv2.ellipse` avec un grand axe égal à la distance entre les deux points, un petit axe égal à la moitié, et l'angle calculé avec `arctan2` pour qu'elle suive la souris.

La Figure 1 montre les formes dessinées dans l'interface : une étoile de lignes dans les 8 directions, un cercle, un rectangle, ect.

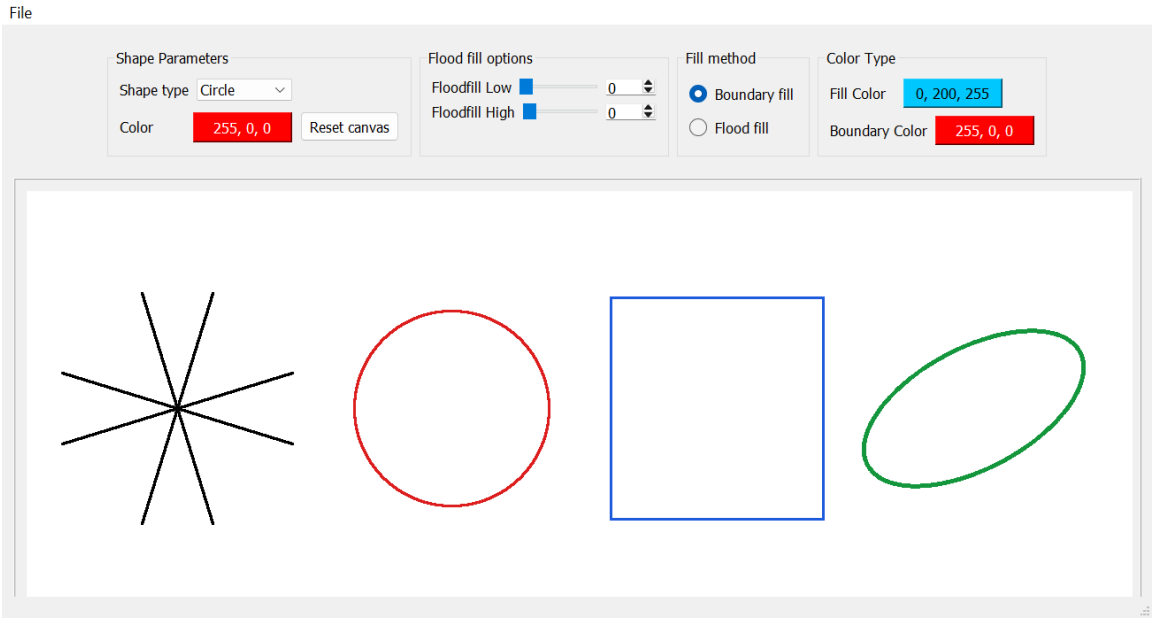


FIGURE 1 – Tracé des quatre primitives dans l'interface : étoile de lignes, cercle, rectangle et ellipse Crédit : JONNIAUX Mathieu

La Figure 2 montre un zoom sur une ligne de 1 pixel d'épaisseur : on voit bien l'effet d'escalier de Bresenham, où chaque pixel touche le suivant en ligne droite ou en diagonale. Dans l'interface, j'ai gardé l'épaisseur de 3 pixels du code de départ parce que les formes se voient mieux à l'écran.

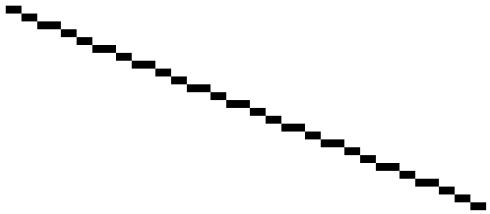


FIGURE 2 – Zoom (×8) sur un segment tracé avec l'algorithme de Bresenham à 1 pixel d'épaisseur Crédit : JONNIAUX Mathieu

3.1.2 Comparaison du rectangle : Bresenham vs OpenCV

Comme demandé dans l'énoncé, j'ai comparé deux méthodes pour le rectangle : tracer 4 lignes de Bresenham ou appeler `cv2.rectangle`. À taille normale dans l'application, il n'y a aucune différence visible. En zoomant fort sur un coin avec l'épaisseur de 3 pixels (Figure 3), mes 4 lignes de Bresenham à gauche allument un carré de 3×3 sur chaque point et font un coin extérieur droit, alors que `cv2.rectangle` à droite coupe le pixel tout au bout de l'angle.



FIGURE 3 – Comparaison d'un coin de rectangle (×8) : 4 segments de Bresenham à gauche, `cv2.rectangle` à droite Crédit : JONNIAUX Mathieu

3.1.3 Amélioration de la Partie 1 : prévisualisation en direct

Dans le code fourni au départ, rien ne s'affichait pendant qu'on bougeait la souris : la forme aparaissait seulement quand on lâchait le clic, ce qui n'était pas pratique pour viser. J'ai donc ajouté un aperçu dans `mouseMoveEvent` qui dessine la forme sur une copie temporaire de l'image tant que le clic gauche est enfoncé, et modifie la vraie image seulement au relâchement.

3.2 Partie 2 : Remplissage automatisé (BoundaryFill et FloodFill)

3.2.1 BoundaryFill récursif et validation sur `test.png`

En suivant le conseil de l'énoncé, j'ai d'abord testé mon algorithme récursif `BoundaryFill` à 4 voisins dans `test_fill.py` sur `images/test.png` à partir du point $(15, 15)$: tant que le pixel n'est ni sur la bordure ni déjà colorié, on le remplit et on rappelle la fonction sur ses 4 voisins. Sur la Figure 4, l'intérieur du carré est rempli en cyan sans toucher ni traverser le bord rouge.

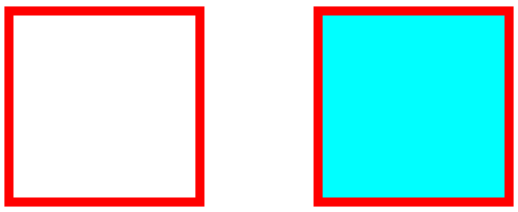


FIGURE 4 – Validation de `BoundaryFill` sur `test.png` : image originale à gauche, résultat rempli en cyan à droite Crédit : JONNIAUX Mathieu

Une fois que ça fonctionnait, j'ai copié ma logique dans `boundary_fill_model.py`. La Figure 5 montre le remplissage sur plusieurs formes dessinées à la souris. J'ai notamment tracé un rectangle coupé en deux par une diagonale de Bresenham : quand on lance le remplissage jaune dans le triangle du haut, il s'arrête sur la ligne oblique parce qu'en voisinage 4, l'algorithme ne peut pas sauter en diagonale par-dessus le trait.

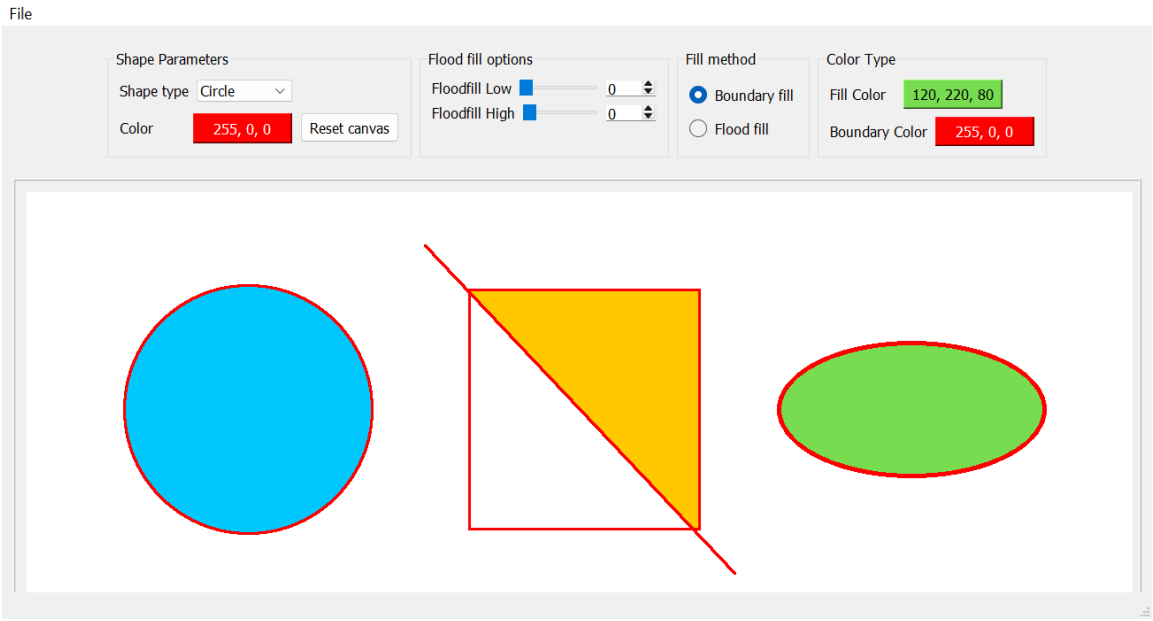


FIGURE 5 – Remplissage `BoundaryFill` de formes dessinées (cercle, demi-rectangle coupé par une diagonale, ellipse) Crédit : JONNIAUX Mathieu

3.2.2 Limite de BoundaryFill sur `cube.png` et passage à FloodFill

Quand on charge une vraie image comme `cube.png` et qu'on essaie de remplir une case avec `BoundaryFill` en mettant du noir $(0, 0, 0)$ comme bordure, la couleur fuit et remplit toute la fenêtre (Figure 6). En regardant les valeurs des pixels de `cube.png`, j'ai vu qu'à cause de l'anticrénelage et de la compression, les traits noirs du cube ne valent presque jamais exactement $(0, 0, 0)$: il n'y a que 630 pixels vraiment à $(0, 0, 0)$, les autres tournent plutôt entre $(35, 0, 0)$ et $(55, 55, 55)$. Comme `BoundaryFill` teste l'égalité exacte avec $(0, 0, 0)$, il passe directement à travers ces pixels gris foncé.

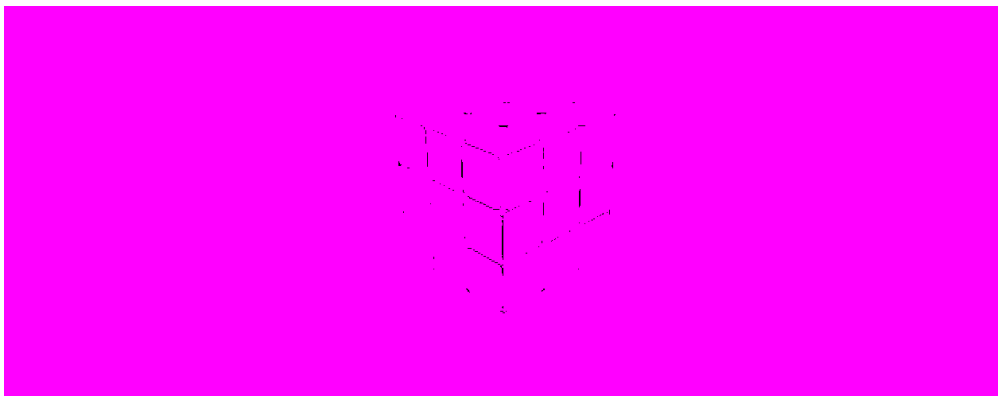


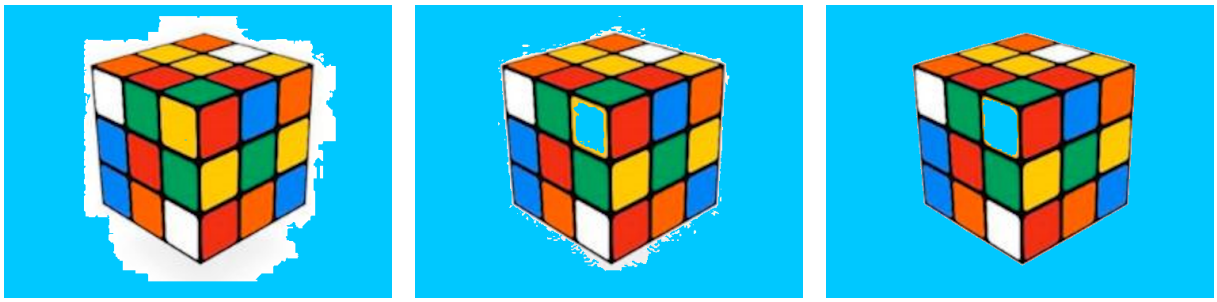
FIGURE 6 – Échec de `BoundaryFill` sur `cube.png` Crédit : JONNIAUX Mathieu

Avec `cv2.floodFill` et l'option `FLOODFILL_FIXED_RANGE`, ça marche beaucoup mieux parce qu'on compare chaque pixel à la couleur du pixel cliqué au départ avec une marge :

$$C_{\text{départ}} - \text{low} \leq C(x, y) \leq C_{\text{départ}} + \text{high} \tag{1}$$

Le seuil *low* permet d'accepter des pixels plus sombres que le point cliqué, et *high* des pixels plus clairs. La Figure 7 montre le résultat sur `cube.png` en cliquant sur une case jaune puis sur le fond blanc :

- low $= 0$, high $= 0$ (Figure 7a) : seuls les pixels strictement identiques changent. Sur la case jaune ça ne colorie presque rien, et sur le fond ça bloque dès le début de l'ombre grise.
- low $= 15$, high $= 15$ (Figure 7b) : la case jaune est remplie d'un seul coup et le fond commence à manger le bord clair de l'ombre.
- low $= 60$, high $= 60$ (Figure 7c) : toute la case jaune et l'ombre grise autour du cube sont remplie en cyan sans déborder sur les contours noirs.



(a) low $= 0$, high $= 0$ (b) low $= 15$, high $= 15$ (c) low $= 60$, high $= 60$

FIGURE 7 – Impact des seuils *low* et *high* de `FloodFill` sur `cube.png` (clic sur une facette jaune et sur le fond blanc) Crédit : JONNIAUX Mathieu

3.3 Partie 3 : Extraction de la silhouette humaine

3.3.1 Logique du pipeline de soustraction de fond

Ici, le programme compare chaque image de la webcam au fond vide enregistré au départ. Comme une simple soustraction donne beaucoup de bruit et de trous, j'ai découpé le traitement en 6 étapes résumées à la Figure 8.

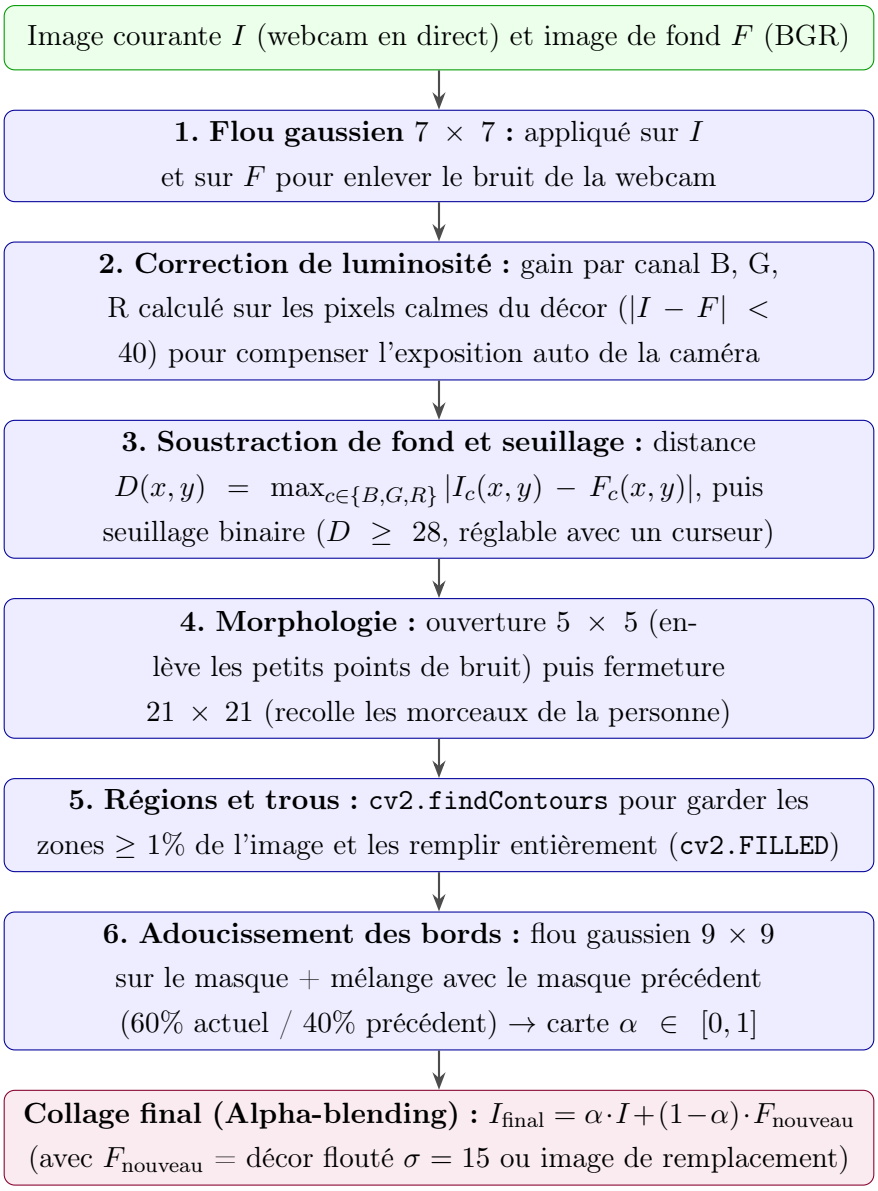


FIGURE 8 – Diagramme de flux du pipeline d'extraction de la silhouette et de remplacement de fond (Partie 3) Crédit : JONNIAUX Mathieu

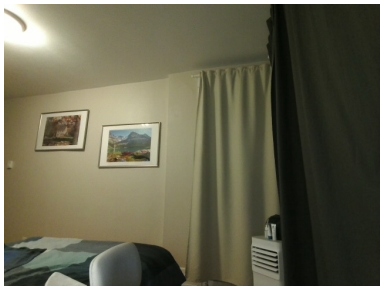
3.3.2 Justification et visualisation des étapes intermédiaires

J'ai enregistré les images de chaque étape pendant mes essais avec ma webcam dans ma chambre (Figure 9) :

— **Flou gaussien** : même sur un mur fixe, les pixels de la webcam bougent un peu d'une image à l'autre. Lisser au départ stabilise la soustraction.

— **Correction de luminosité** : quand j'arrive devant la caméra avec mon T-shirt noir, la webcam ajuste toute seule son exposition et le mur derrière moi change de luminosité par rapport à la photo innitiale. Ajuster la moyenne des canaux B, G et R sur les zones calmes du mur (avec un gain borné entre 0.75 et 1.35 pour éviter les valeurs extrêmes) évite que tout le mur soit détecté.

— **Différence sur les 3 canaux** : passer en niveaux de gris faisait perdre du contraste sur le visage, donc je garde le plus grand écart parmi B, G et R (Figure 9c). Après le seuillage binaire (Figure 9d), il reste encore quelques petits trous sur la main et les bords.
— **Ouverture puis fermeture** : l'ouverture supprime les petits points blancs isolés dans le fond, et une grande fermeture recolle les zones proches sur le visage et les doigts (Figure 9e).
— **Contours et remplissage plein** : en redessinant en plein (`cv2.FILLED`) seulement les contours assez grands, tous les trous qui restaient au milieu du corps disparaissent d'un coup (Figure 9f).
— **Adoucissement et mélange temporel** : adoucir le masque au flou gaussien et le mélanger avec celui de l'image précédente (Figure 9g) enlève l'effet d'escalier autour des cheveux et empêche le contour de trembler en direct.



(a) Fond de référence F



(b) Image courante I



(c) Distance $\max |I - F|$



(d) Masque brut après seuillage ($D \geq 28$)



(e) Après ouverture et fermeture



(f) Après tri des régions et remplissage



(g) Carte α adoucie + mélange temporel

FIGURE 9 – Étapes intermédiaires de la Partie 3 capturées lors de mes essais avec ma webcam Crédit : JONNIAUX Mathieu

3.3.3 Résultats finaux : fond flouté et fond remplacé

Les Figures 10 et 11 montrent le rendu final en direct avec ma webcam. Pour le mode « Blur background » (Figure 10), j'applique un gros flou gaussien sur l'image du fond : mon visage et ma main restent nets pendant que l'arrière-plan de ma chambre (les cadres, les rideaux, ect.) est flouté. Pour le mode « Replace background » (Figure 11), le fond est remplacé par une photo de cerisiers japonais redimensionnée en gardant les proportions.



FIGURE 10 – Résultat final de la Partie 3 avec le Blur sur ma webcam Crédit : JONNIAUX Mathieu



FIGURE 11 – Résultat final de la Partie 3 avec Replace background sur ma webcam Crédit : JONNIAUX Mathieu

4 Difficultés et améliorations

4.1 Difficultés rencontrées et limites

— **Dépassement de la récursion (`RecursionError`) dans BoundaryFill :** le canevas faisant $1250 \times 490 = 612\,500$ pixels, un clic dans le vide ou une fuite sur `cube.png` dépasse vite la limite `sys.setrecursionlimit(100000)`. Mon premier essai de secours avec une pile s'arrêtait tout de suite car le pixel de départ avait déjà été colorié par le début de la récursion. J'ai corrigé ça en ajoutant un tableau booléen `visite` pour retraverser la zone déjà coloriée et finir le remplissage.

— **Comprendre l'effet de *low* et *high* sur un fond blanc :** lors de mes premiers tests sur `cube.png`, j'avais monté le slider *High* très haut (à 197) en laissant *Low* proche de 0, et le remplissage du fond blanc ne bougeait pas. En reprenant l'équation (1), j'ai compris que sur un pixel blanc $(255, 255, 255)$, on est déjà au maximum possible : monter *High* ne sert à rien, et c'est seulement *Low* qui permet d'aller chercher les pixels plus foncés comme l'ombre grise. En plus, le slider *Low* de l'interface de départ était bloqué à 50 maximum, donc je l'ai passé à 255.

— **Le seuillage d'Otsu piégé par mon T-shirt noir :** dans ma première version de la Partie 3, j'avais utilisé Otsu pour choisir le seuil automatiquement. Ça marchait bien sur la vidéo fournie, mais devant ma webcam avec un T-shirt noir et mon mur clair, mon visage était presque entièrement flouté. En mesurant les écarts, mon T-shirt noir faisait une différence d'environ 170 avec le mur, contre seulement 25 à 45 pour la peau de mon visage. Comme Otsu coupe l'histogramme en deux groupes, le gros pic du T-shirt tirait le seuil vers 70 et mettait mon visage dans le fond. Revenir à un seuil simple réglé à 28 avec un slider a réglé le problème.

— **Essai avec le filtre de Sobel :** comme le gradient de Sobel m'avait beaucoup aidé au laboratoire 2, j'ai essayé de comparer aussi les contours de Sobel entre l'image et le fond pour mieux détecter un pull clair devant un mur clair. En pratique ça ne détectait que le bord extérieur de la personne et ça rajoutait plein de faux positifs sur les cadres dès que la caméra vibrait un peu, donc j'ai enlevé cette idée.

4.2 Améliorations apportées et proposées

Améliorations que j'ai ajoutées à l'application :

— **Champs numériques (`QSpinBox`) et sauvegarde des couleurs :** j'ai ajouté des `QSpinBox` de 0 à 255 à côté des sliders pour taper une valeur précise, gardé en mémoire la dernière couleur choisie et affiché la couleur directement sur les boutons.

— **Aperçu en direct pendant le tracé :** voir section 3.1.3.

— **Interface webcam plus pratique :** la vidéo avait une hauteur bloquée à 360 pixels ; j'ai modifié le layout avec `QSizePolicy.Ignored` pour qu'elle s'agrandisse avec la fenêtre. J'ai aussi ajouté un slider pour le seuil et une case « Voir masque » pour contrôler le masque en direct.

Améliorations futures possibles (toujours sans apprentissage profond) :

— **Remplissage par lignes (*Run-fill / Scanline*) :** remplir toute une ligne horizontale d'un coup au lieu d'empiler chaque pixel dans `BoundaryFill` pour aller plus vite et éviter de saturer la pile.

— **Moyenne du fond sur plusieurs images :** moyenner une trentaine d'images au moment d'enregistrer le fond pour estimer le bruit de chaque pixel et adapter le seuil localement.

— **Filtrer les ombres en HSV ou LAB :** quand je m'approche du mur, mon ombre assombrit le mur et finit parfois dans le masque. En passant en HSV ou LAB comme au laboratoire 1, on pourrait vérifier si seule la luminosité a baissé sans que la teinte change pour ignorer l'ombre.

5 Conclusion

Ce troisième laboratoire m'a permis de coder les algorithmes de base du dessin matriciel : tracer des lignes et des cercles avec des calculs entiers (Bresenham), comprendre pourquoi le voisinage 4 est indispensable pour le remplissage, et voir pourquoi `FloodFill` avec tolérance marche beaucoup mieux que `BoundaryFill` sur une image réelle comme `cube.png`. C'est la partie 3 sur la webcam (le flou et le remplacement de fond) qui m'a pris le plus de temps à régler à cause des changements d'exposition et du T-shirt noir, mais c'est aussi celle qui m'a le plus plu car on voit directement le résultat sur soi en temps réel.

Références

OpenCV : *Open Source Computer Vision Library Documentation*, en ligne : https://docs.opencv.org/.

NumPy Developers : *NumPy Reference Manual*, en ligne : https://numpy.org/doc/.

Riverbank Computing : *PyQt5 Reference Guide*, en ligne : https://www.riverbankcomputing.com/static/Docs/PyQt5/.

Vázquez, C. (2026) : *Notes de cours GTI411 – Imagerie numérique (C01 à C09)*, École de technologie supérieure (ÉTS), Montréal.